EDGE INTELLIGENCE – LAB ASSIGNMENT – 19-2-26

NAME: SHYAM.K

REG.NO: 25MML0040

CODE:

```
[ ]
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
```

```
[ ]
# Load pretrained MobileNetV2 (re-initializing model variable)
model = MobileNetV2(weights='imagenet')
```

```
[ ]
import cv2
import numpy as np

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)

    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    # Handle videos where frame count is 0
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
```

```python
    if total_frames > 0:
        step = max(1, total_frames // max_frames)
    else:
        step = 1

    count = 0
    debug_printed = False

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  # 🔴 Important fix
            frames.append(frame)

        count += 1

        if len(frames) >= max_frames:
            break

    cap.release()
    print("Extracted frames:", len(frames))
    return np.array(frames)
```

```python
def classify_video(video_path):
    frames = extract_frames(video_path)

    if len(frames) == 0:
        print("No frames extracted!")
        return

    frames_processed = preprocess_input(frames)
    predictions = model.predict(frames_processed)

    # Average predictions across frames
    avg_pred = np.mean(predictions, axis=0)
    top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    for pred in top_pred[0]:
      print(pred[1], ":", round(pred[2]*100, 2), "%")

    print("Final Prediction:", top_pred[0][0][1])
    return frames # Return the extracted frames

# Replace with your video path
video_path = "/content/drive/MyDrive/Copy of sample_video.mp4"
# Call classify_video and store the frames
extracted_frames_for_plotting = classify_video(video_path)
```

```
Extracted frames: 20
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 885ms/step
digital_clock : 30.81 %
screen : 6.22 %
monitor : 5.01 %
web_site : 4.31 %
analog_clock : 3.65 %
Final Prediction: digital_clock
```

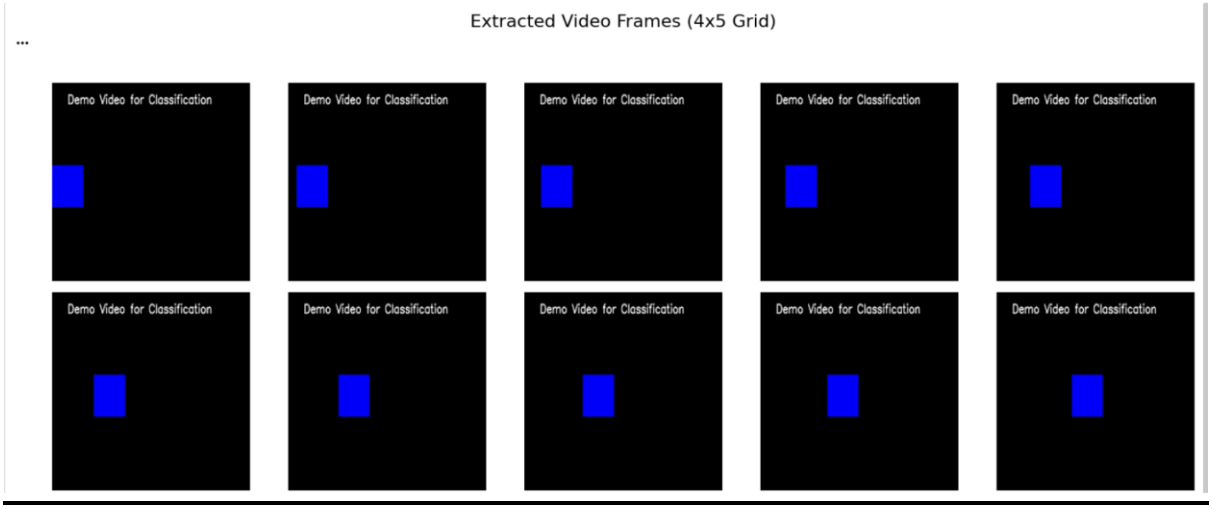
```python
import matplotlib.pyplot as plt

# Use the frames extracted by the classify_video function
# The variable 'extracted_frames_for_plotting' now holds the frames

if len(extracted_frames_for_plotting) > 0:
    # Create a figure and a set of subplots for displaying the frames
    fig, axes = plt.subplots(4, 5, figsize=(15, 12))
    axes = axes.flatten()

    # Iterate through the extracted_frames array and display each frame
    for i, frame in enumerate(extracted_frames_for_plotting):
        axes[i].imshow(frame) # No need for normalization here if extract_frames returns [0
        axes[i].axis('off')  # Turn off axis labels and ticks

    plt.suptitle("Extracted Video Frames (4x5 Grid)", fontsize=16)
    plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust layout to prevent overlapping and mal
    plt.show()
else:
    print("No frames were extracted to display.")
```



```python
print("\nRaw pixel values for the first extracted frame (index 0):")
print(extracted_frames_for_plotting[0])
print("\nShape of the first frame:", extracted_frames_for_plotting[0].shape)
print("\nData type of pixel values:", extracted_frames_for_plotting[0].dtype)
```

```
Raw pixel values for the first extracted frame (index 0):
[[[0 0 0]
  [0 0 0]
  [0 0 0]
  ...
  [0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]
  ...
  [0 0 0]
  [0 0 0]
  [0 0 0]]

 [[0 0 0]
  [0 0 0]
  [0 0 0]
  ...
  [0 0 0]
  [0 0 0]
  [0 0 0]]
```

LEARNING OUTCOMES:

1. Video Processing Fundamentals: We learned how to extract frames from a video file using OpenCV, demonstrating basic video handling and frame manipulation (resizing, color conversion).
2. Image Classification with Pre-trained Models: We gained experience in applying a pre-trained deep learning model (MobileNetV2) from TensorFlow/Keras for image classification, including preprocessing images and interpreting model predictions.
3. Data Visualization for Analysis: We utilized Matplotlib to visualize extracted video frames, which is a fundamental skill for inspecting image-based data and verifying processing steps.